L21 - 部署与应用

> **"从实验室走向生产"**

本节目标

1. 掌握模型格式转换流程：PyTorch → Transformers → GGUF
2. 学会使用 ollama、vLLM、llama.cpp 部署 MiniMind
3. 理解 MiniMind 的 OpenAI API 兼容接口
4. 了解 Tool Calling 和 Adaptive Thinking 功能
5. 了解 Streamlit WebUI 和第三方 UI 接入
6. 了解 DDP 多卡训练的基本概念

前置知识

- L20 推理优化（KV-Cache、生成策略）
- 基本的命令行操作
- 了解 HTTP API 的概念
- 了解 Docker 基本概念（可选）

一、模型格式转换

1.1 为什么需要格式转换

MiniMind 训练时使用 PyTorch 原生格式（`.pth` 文件），但不同推理框架有各自的格式要求：

```
PyTorch (.pth)
    ↓ 转换
Transformers (HuggingFace 格式)
    ↓ 转换
GGUF (llama.cpp / ollama 格式)
```

1.2 PyTorch → Transformers

Transformers 格式是 HuggingFace 生态的标准，包含：

```
model_directory/
├── config.json              # 模型配置
├── model.safetensors        # 模型权重（safetensors 格式）
├── tokenizer.json           # 分词器
├── tokenizer_config.json    # 分词器配置
└── generation_config.json   # 生成配置
```

MiniMind 提供了转换脚本，核心逻辑：

```
# 加载 PyTorch 模型
state_dict = torch.load("minimind.pth", map_location="cpu")

# 映射权重名称到 Transformers 格式
mapping = {
    "tok_embeddings.weight": "model.embed_tokens.weight",
    "layers.{}.attention.wq.weight": "model.layers.{}.self_attn.q_proj.weight",
    "layers.{}.attention.wk.weight": "model.layers.{}.self_attn.k_proj.weight",
    "layers.{}.attention.wv.weight": "model.layers.{}.self_attn.v_proj.weight",
    # ... 更多映射
}

# 保存为 Transformers 格式
model.save_pretrained("minimind-hf")
tokenizer.save_pretrained("minimind-hf")
```

1.3 Transformers → GGUF

GGUF（GPT-Generated Unified Format）是 llama.cpp 项目定义的模型格式，支持各种量化级别：

```
# 使用 llama.cpp 提供的转换脚本
python convert_hf_to_gguf.py minimind-hf/ --outfile minimind.gguf --outtype f16

# 进一步量化
./llama-quantize minimind.gguf minimind-Q4_K_M.gguf Q4_K_M
```

常见量化格式： | 格式 | 每参数位数 | 精度损失 | 适用场景 | |------|----------|---------|--------- | | F16 | 16 bit | 无 | 精度优先 | | Q8_0 | 8 bit | 极小 | 平衡 | | Q5_K_M | ~5.5 bit | 小 | 推荐 | | Q4_K_M | ~4.5 bit | 中等 | 速度优先 | | Q2_K | ~2.5 bit | 较大 | 极端压缩 |

二、ollama 部署

2.1 什么是 ollama

ollama 是一个简单易用的本地 LLM 运行工具，类似于 Docker 的操作体验——拉取模型，一键运行。

2.2 直接运行 MiniMind

```
# MiniMind 已经上传到 ollama 模型库
ollama run jingyaogong/minimind-3
```

就这么简单！ollama 会自动下载模型文件并启动对话界面。

2.3 自定义 Modelfile

如果你想自定义推理参数，可以创建 Modelfile：

```
FROM minimind.gguf

PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER top_k 50
PARAMETER repeat_penalty 1.1

SYSTEM """你是MiniMind，一个小型但有用的AI助手。"""
```

然后构建并运行：

```
ollama create my-minimind -f Modelfile
ollama run my-minimind
```

2.4 ollama API

ollama 默认启动在 `localhost:11434`，提供 REST API：

```
# 生成回答
curl http://localhost:11434/api/generate -d '{
  "model": "jingyaogong/minimind-3",
  "prompt": "什么是机器学习？",
  "stream": false
}'
```

```
# 对话模式
curl http://localhost:11434/api/chat -d '{
  "model": "jingyaogong/minimind-3",
  "messages": [
    {"role": "user", "content": "你好"}
  ]
}'
```

三、vLLM 部署

3.1 什么是 vLLM

vLLM 是一个高吞吐量的 LLM 推理引擎，专为**服务端部署**设计。核心技术是 **PagedAttention**——把 KV-Cache 像操作系统管理虚拟内存一样分页管理，大幅提高 GPU 内存利用率和并发请求处理能力。

3.2 vLLM 的核心优势

1. **PagedAttention**：解决 KV-Cache 内存碎片问题，提高显存利用率
2. **连续批处理（Continuous Batching）**：动态地将新请求插入正在处理的 batch，不用等当前 batch 全部完成
3. **高吞吐**：比朴素实现快 2-24 倍
4. **OpenAI 兼容 API**：可以直接替换 OpenAI 的 API 接口

3.3 使用 vLLM 部署 MiniMind

```
# 安装 vLLM
pip install vllm

# 启动推理服务（需要 Transformers 格式的模型）
python -m vllm.entrypoints.openai.api_server \
    --model minimind-hf/ \
    --host 0.0.0.0 \
    --port 8000
```

然后就可以用 OpenAI 兼容的 API 访问：

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="none")

response = client.chat.completions.create(
    model="minimind-hf",
    messages=[{"role": "user", "content": "你好"}],
```

```
    temperature=0.7,
)
print(response.choices[0].message.content)
```

四、llama.cpp 部署

4.1 什么是 llama.cpp

llama.cpp 是一个纯 C/C++ 实现的 LLM 推理框架，可以在 **CPU 上**高效运行大语言模型，不需要 GPU。

4.2 核心优势

1. **纯 CPU 推理**：不需要 GPU，笔记本电脑也能跑
2. **多种量化**：支持 Q2 到 Q8 各种量化级别
3. **跨平台**：支持 Linux、macOS、Windows、Android
4. **低依赖**：不需要 Python、PyTorch 等

4.3 使用 llama.cpp 运行 MiniMind

```
# 编译 llama.cpp
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp && make

# 运行（需要 GGUF 格式模型）
./llama-cli -m minimind-Q4_K_M.gguf \
    -p "请介绍一下人工智能" \
    -n 256 \
    --temp 0.7
```

4.4 vLLM vs llama.cpp

维度	vLLM	llama.cpp
硬件需求	GPU	CPU（也支持 GPU）
并发处理	优秀（Continuous Batching）	较弱
适用场景	高并发服务端	个人设备、边缘部署
速度	极快（GPU 加速）	较慢（CPU 受限）
模型格式	HuggingFace	GGUF

维度	vLLM	llama.cpp
量化支持	GPTQ/AWQ	GGUF 内置量化

五、MiniMind OpenAI 兼容接口

5.1 serve_openai_api.py

MiniMind 自带一个 OpenAI API 兼容的服务脚本 `serve_openai_api.py`，使用 FastAPI 实现：

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatMessage(BaseModel):
    role: str
    content: str

class ChatRequest(BaseModel):
    model: str
    messages: list[ChatMessage]
    temperature: float = 0.7
    top_p: float = 0.9
    max_tokens: int = 512
    stream: bool = False

@app.post("/v1/chat/completions")
async def chat_completions(request: ChatRequest):
    # 构建 prompt
    prompt = build_prompt(request.messages)

    # 模型推理
    response = model.generate(
        prompt,
        temperature=request.temperature,
        top_p=request.top_p,
        max_tokens=request.max_tokens
    )

    # 返回 OpenAI 格式的响应
    return {
        "id": f"chatcmpl-{uuid4()}",
        "object": "chat.completion",
        "model": request.model,
        "choices": [{
            "index": 0,
            "message": {"role": "assistant", "content": response},
            "finish_reason": "stop"
```

```
        }]
    }
```

5.2 启动服务

```
python serve_openai_api.py --model_path out/minimind --port 8080
```

启动后可以用任何支持 OpenAI API 的客户端连接：

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8080/v1", api_key="none")
response = client.chat.completions.create(
    model="minimind",
    messages=[{"role": "user", "content": "1+1=?"}]
)
```

六、Streamlit WebUI

6.1 web_demo.py

MiniMind 提供了基于 Streamlit 的 Web 演示界面：

```
streamlit run web_demo.py
```

打开浏览器访问 `http://localhost:8501`，可以进行对话。

6.2 核心实现

```
import streamlit as st

st.title("MiniMind Chat")

# 会话历史
if "messages" not in st.session_state:
    st.session_state.messages = []

# 显示历史消息
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# 用户输入
```

```
if prompt := st.chat_input("请输入你的问题"):
    st.session_state.messages.append({"role": "user", "content": prompt})

    with st.chat_message("user"):
        st.write(prompt)

    # 模型生成
    with st.chat_message("assistant"):
        response = model.generate(prompt)
        st.write(response)

    st.session_state.messages.append({"role": "assistant", "content": response})
```

七、Tool Calling 功能

7.1 什么是 Tool Calling

Tool Calling（工具调用）让 LLM 不仅能生成文本，还能调用外部工具（如搜索引擎、计算器、API 等）。

```
用户: 今天北京天气怎么样?
模型: [调用天气API] → 获取结果 → 北京今天晴，25度
```

7.2 MiniMind 的 Tool Calling 实现

MiniMind 使用特殊标签来标识工具调用:

```
用户消息: 帮我查一下北京的天气

模型输出:
<tool_call>
{"name": "get_weather", "arguments": {"city": "北京"}}
</tool_call>

系统执行工具后返回:
<tool_response>
{"temperature": 25, "weather": "晴", "humidity": 40}
</tool_response>

模型继续生成:
北京今天天气晴朗，气温25度，湿度40%，非常适合出门。
```

7.3 工具定义格式

在系统消息中定义可用工具:

```
{
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "获取指定城市的天气信息",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "城市名称"
            }
          },
          "required": ["city"]
        }
      }
    }
  ]
}
```

7.4 eval_toolcall.py 示例

MiniMind 提供了 `eval_toolcall.py` 来演示和评估 Tool Calling 功能：

```
# 定义工具
tools = [
    {
        "name": "calculator",
        "description": "进行数学计算",
        "parameters": {"expression": "string"}
    },
    {
        "name": "search",
        "description": "搜索信息",
        "parameters": {"query": "string"}
    }
]

# 构建包含工具定义的 prompt
system_prompt = f"你可以使用以下工具：{json.dumps(tools, ensure_ascii=False)}"

# 模型生成
response = model.generate(system_prompt + "\n" + user_query)

# 解析 <tool_call> 标签
if "<tool_call>" in response:
    tool_call = extract_tool_call(response)
    # 执行工具
    tool_result = execute_tool(tool_call)
    # 将结果注入并继续生成
```

```
    response += f"\n<tool_response>{json.dumps(tool_result)}</tool_response>\n"
    final_response = model.generate(response)
```

7.5 Tool Calling 的训练

Tool Calling 能力不是天生的，需要在 SFT 阶段用包含工具调用的训练数据来教会模型：

1. 识别何时需要调用工具
2. 生成正确的 `<tool_call>` 标签和 JSON 参数
3. 理解 `<tool_response>` 中的结果
4. 基于工具结果生成最终回答

八、Adaptive Thinking（自适应思考）

8.1 什么是 Adaptive Thinking

受 OpenAI o1 等推理模型的启发，MiniMind 实现了自适应思考功能。模型在回答前先在 `<think>` 标签中进行"思考"：

```
用户: 123 × 456 等于多少?

模型输出:
<think>
让我计算一下...
123 × 456
= 123 × 400 + 123 × 56
= 49200 + 6888
= 56088
</think>

123 × 456 = 56088
```

8.2 open_thinking 开关

MiniMind 提供 `open_thinking` 参数来控制是否启用思考模式：

```
# 开启思考模式
response = model.generate(prompt, open_thinking=True)

# 关闭思考模式
response = model.generate(prompt, open_thinking=False)
```

- **开启时**：模型会先输出 `<think>...</think>` 进行推理，然后给出答案

- **关闭时**：模型直接输出答案，不做显式思考

8.3 实现原理

```
def generate_with_thinking(model, prompt, open_thinking=True):
    if open_thinking:
        # 在 prompt 后追加 <think> 标签，引导模型开始思考
        prompt = prompt + "<think>\n"

    response = model.generate(prompt)

    if open_thinking:
        # 解析思考内容和最终回答
        think_content, answer = parse_thinking(response)
        return think_content, answer
    else:
        return None, response
```

九、接入第三方 UI

9.1 FastGPT

FastGPT 是一个开源的知识库问答系统，支持通过 OpenAI API 接入自定义模型：

```
# FastGPT 配置
OPENAI_BASE_URL=http://localhost:8080/v1
CHAT_API_KEY=none
```

9.2 Open-WebUI

Open-WebUI（前 Ollama-WebUI）提供了美观的 ChatGPT 风格界面：

```
# 通过 Docker 启动
docker run -d --name open-webui \
    -p 3000:8080 \
    -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
    ghcr.io/open-webui/open-webui:main
```

配置好后，选择 MiniMind 模型即可进行对话。

十、DDP 多卡训练简介

10.1 什么是 DDP

DDP（Distributed Data Parallel）是 PyTorch 的分布式数据并行训练方式。每张 GPU 持有模型的完整副本，但处理不同的数据子集。

10.2 DDP 的工作原理

```
GPU 0: 模型副本 → 数据子集 A → 计算梯度 ─┐
GPU 1: 模型副本 → 数据子集 B → 计算梯度 ─┤ → AllReduce → 平均梯度 → 各自更新
GPU 2: 模型副本 → 数据子集 C → 计算梯度 ─┤
GPU 3: 模型副本 → 数据子集 D → 计算梯度 ─┘
```

10.3 MiniMind 的 DDP 训练

```
# 使用 torchrun 启动 DDP 训练
torchrun --nproc_per_node=4 train.py \
    --use_ddp \
    --batch_size 32
```

核心代码变更：

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

# 初始化分布式环境
dist.init_process_group(backend='nccl')
local_rank = int(os.environ['LOCAL_RANK'])
torch.cuda.set_device(local_rank)

# 包装模型
model = model.to(local_rank)
model = DDP(model, device_ids=[local_rank])

# 数据 Sampler
sampler = DistributedSampler(dataset)
dataloader = DataLoader(dataset, sampler=sampler)
```

10.4 DDP 的注意事项

1. **batch_size**：每张 GPU 的 local batch_size × GPU 数量 = 全局 batch_size
2. **学习率**：通常全局 batch_size 增大时需要相应增大学习率（线性缩放法则）
3. **随机种子**：各 GPU 的数据采样不同，但模型初始化要一致
4. **保存模型**：只在 rank 0 保存模型

🎤 面试考点

Q1: 模型部署有哪些常见方案？各适用什么场景？

答： - **ollama**：简单易用，适合个人本地使用和快速原型 - **vLLM**：高吞吐，适合服务端高并发部署，需要 GPU - **llama.cpp**：纯 CPU 推理，适合无 GPU 环境、边缘设备、移动端 - **自建 API 服务**：灵活定制，适合特定业务需求

Q2: vLLM 和 llama.cpp 的核心区别是什么？

答： vLLM 面向**高并发 GPU 推理**，核心技术是 PagedAttention 和 Continuous Batching，吞吐量极高；llama.cpp 面向**低资源 CPU 推理**，核心是高效的 C++ 实现和多种量化格式支持。前者适合服务端，后者适合端侧。

Q3: Tool Calling 是怎么实现的？

答：Tool Calling 通过在 SFT 阶段训练模型识别需要调用工具的场景，输出包含 `<tool_call>` 标签的结构化 JSON。外部系统解析 JSON 执行工具后，将结果通过 `<tool_response>` 标签注入上下文，模型基于工具结果继续生成最终回答。本质上是通过特殊标记实现的 structured generation。

Q4: 什么是 GGUF 格式？为什么需要它？

答：GGUF 是 llama.cpp 定义的统一模型格式，支持在文件中嵌入各种量化的权重、模型配置和分词器信息。需要它是因为 llama.cpp 是 C++ 实现，无法直接加载 PyTorch 格式的模型。GGUF 格式还支持 mmap 加载，启动速度快。

Q5: PagedAttention 是什么？解决什么问题？

答：PagedAttention 是 vLLM 的核心技术，将 KV-Cache 分成固定大小的"页"，像操作系统管理虚拟内存一样管理 KV-Cache。它解决了传统 KV-Cache 预分配导致的内存碎片和浪费问题，显著提高了 GPU 显存利用率，使得同样的显存可以处理更多并发请求。

Q6: DDP 和模型并行有什么区别？

答：DDP（数据并行）是每张 GPU 持有完整的模型副本，处理不同数据子集，用 AllReduce 同步梯度。模型并行是把模型切分放到不同 GPU 上（Tensor Parallel 切层内参数、Pipeline Parallel 切层间）。DDP 适用于模型能放进单卡显存的场景；模型并行适用于模型太大放不下单卡的场景。

✅ 自测题

1. **填空**：MiniMind 的模型格式转换路径是 PyTorch → _ → _。
2. **判断**：llama.cpp 只能在 CPU 上运行。（对/错）
3. **选择**：以下哪个不是 vLLM 的核心技术？
4. A. PagedAttention
5. B. Continuous Batching
6. C. FlashAttention
7. D. KV-Cache 分页管理
8. **简答**：MiniMind 的 Tool Calling 使用了哪些特殊标签？工作流程是什么？
9. **操作题**：写出使用 ollama 运行 MiniMind 的命令。

▶ 参考答案

下一节预告

下一节是本教程的最后一课——**面试通关指南**。我们将系统梳理如何把 MiniMind 项目写进简历、面试中如何介绍、面试官最常问的 Top 20 问题的标准答案，以及深挖追问的应对策略。